



SECD2523

DataBase

Project: Project Phase 3

Title: Car Equipment Shop Database System

The team for this project consists of the following members:

- ❖ **Dina Mohammed**
- ❖ **Dyako Abubakr**
- ❖ **Mohammed Kamal**

Phase 3: Database System Design and Implementation:

1. Proposal & Database Requirement

Overview of the Project

The car equipment shop project involves developing a robust database system that facilitates efficient inventory management, detailed tracking of sales transactions, and specialized handling of inter-shop borrow/lend operations. The system will cater to a niche market, focusing primarily on Kia and Hyundai car parts, accommodating both retail customers and relationships with other similar businesses.

Problem Statement

The shop currently faces several critical challenges:

- **Inventory Management:** Difficulty in maintaining accurate and up-to-date records of parts in stock, leading to frequent overstocks or stockouts.
- **Sales Transaction Management:** Inefficiencies in tracking sales transactions that complicate financial reporting and customer service.
- **Inter-Shop Transactions:** No systematic approach to manage the borrowing and lending of parts with other shops, leading to potential losses and accounting discrepancies.

Proposed Solutions

To address these issues, we propose the following solutions integrated into the new database system:

- **Real-Time Inventory Tracking:** Implement a dynamic inventory management system that updates in real time as sales are made and parts are received or borrowed/lent.
- **Automated Sales Tracking:** Develop a system to automatically track all sales transactions, including detailed information on parts sold, sales staff involved, and customer data.
- **Inter-Shop Transaction Module:** Create a specialized module to handle borrow/lend transactions between shops, including tracking of borrowed parts, pending payments, and receivables.

Objectives

- **Improve Inventory Accuracy:** Ensure the inventory levels reflected in the system are always current and accurate.
- **Enhance Sales Reporting:** Enable detailed reporting on sales transactions to support financial analysis and customer relationship management.
- **Streamline Inter-Shop Transactions:** Provide a clear and accountable method for managing transactions between shops that involve borrowing and lending parts.

Scopes

- **Inventory Management:** The system will manage all parts inventory, specifically for Kia and Hyundai brands, including categorization by type, model, and other relevant attributes.
- **Sales Management:** It will handle all aspects of sales transactions, from point-of-sale data entry to generating sales reports and managing customer information.
- **Inter-Shop Borrow/Lend System:** The system will include functionality to record and track all aspects of inter-shop transactions, ensuring accurate tracking of parts and financial obligations between parties.
- **User Access Levels:** Different levels of user access will be implemented, from administrative access for managers to basic access for sales personnel.
- **Integration Capability:** The system will be designed to integrate potentially with online platforms and other existing systems like CRM software.

Detailed Description of Implementation

The project will be divided into several phases, starting with the requirements gathering to detailed system design, development, testing, and deployment. We will use modern database management systems (DBMS) and programming practices to ensure reliability and scalability. The database will be normalized to ensure efficiency and integrity of the data. Reporting capabilities will be built using SQL queries that provide real-time insights into business operations.

By addressing these components, the proposed database system will not only solve the existing problems but also provide a platform for future expansion and integration with online sales channels. The project is anticipated to enhance operational efficiency, improve customer satisfaction, and enable smoother interactions with other businesses in the industry.

2. Database Conceptual Design

This section outlines the conceptual design of the database for your car equipment shop, including an Entity-Relationship Diagram (ERD), updated business rules, an enhanced version of the ERD, and a comprehensive data dictionary.

Detailed Explanation of Entities and Relationships

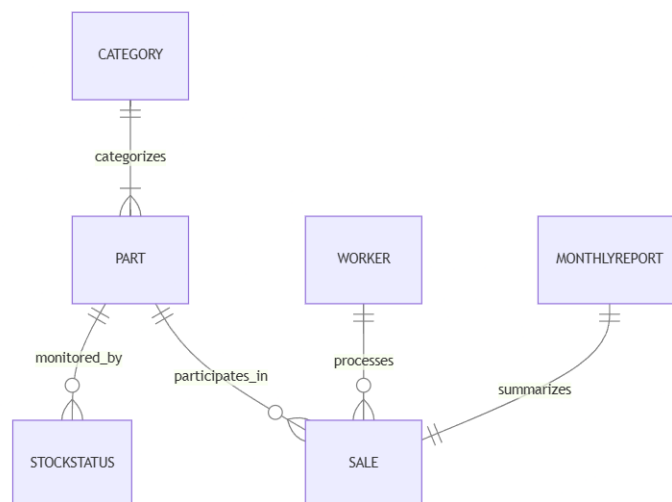
Entities:

1. **Part:** Represents individual items or car parts available in the inventory.
2. **Category:** Groups parts into categories based on their type or function.
3. **Sale:** Transaction records where one or more parts are sold.
4. **Worker:** Employees who handle sales and possibly other transactions.
5. **Transaction:** Records the lending or borrowing of parts, not necessarily with other shops since that table isn't specified but could involve general external transactions.
6. **Monthly Report:** Aggregates sales data over a month, summarizing total sales and revenue.
7. **Stock Status:** Tracks the availability and quantity of each part in stock.

Relationships:

- **Part belongs to a Category:** Each part is associated with one category, helping organize inventory effectively.
- **Sale involves one or more Parts and is handled by a Worker:** Sales transactions include various parts and are processed by workers, capturing who sold what.
- **Part has a Stock Status:** Each part has an associated stock status that tracks its availability and stock levels.

Here's a simple textual description of the ERD:

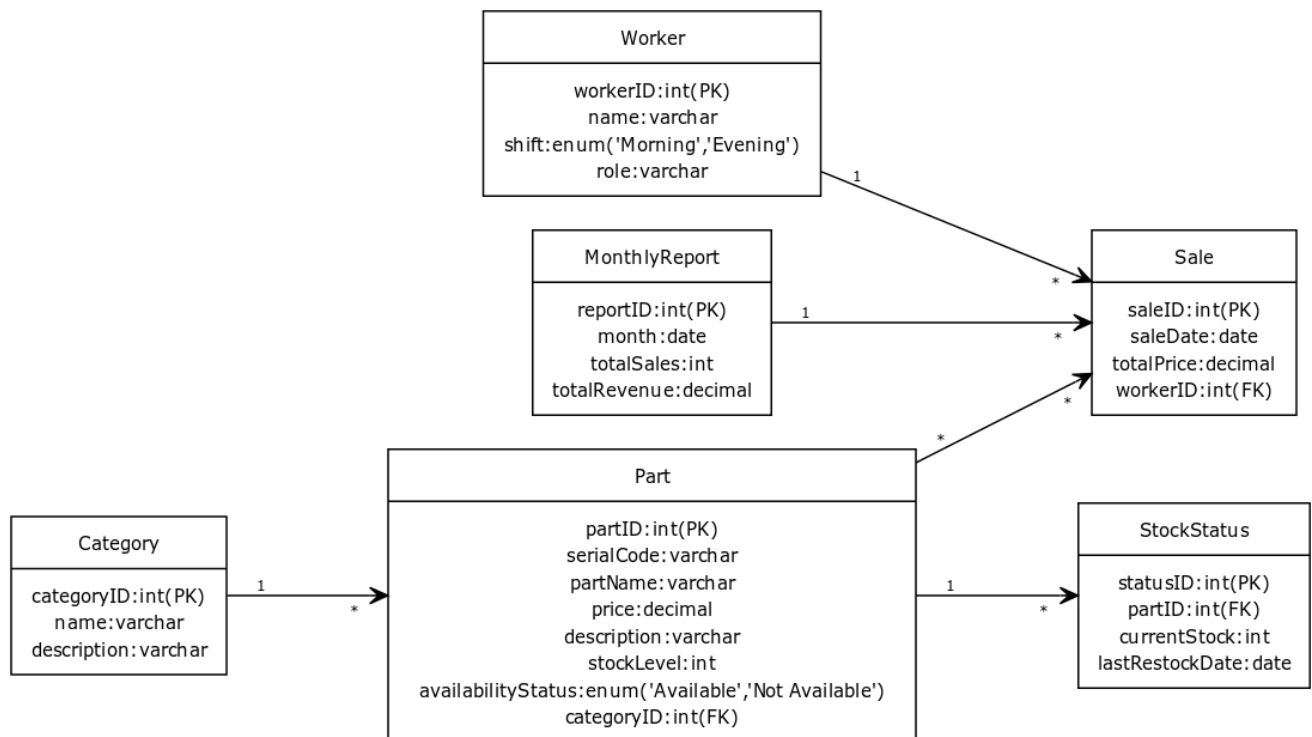


Updated Business Rules

1. **Part**
 - Each part must be linked to exactly one category.
 - Parts can participate in multiple sales.
 - Each part's stock status is tracked individually.
2. **Category**
 - A category groups multiple parts.
3. **Sale**
 - A sale can include multiple parts.
 - Each sale is handled by exactly one worker.
4. **Worker**
 - Workers may process multiple sales.
5. **Monthly Report**
 - Each monthly report summarizes the sales for that month.
6. **Stock Status**
 - Tracks the availability and stock levels of each part.

Enhanced ERD

The enhanced ERD includes not only basic entities and relationships but also attributes and key constraints. Here is an example of how to describe it:



Data Dictionary

Category

- `categoryID`: Primary key, unique identifier for each category.
- `name`: Name of the category.
- `description`: Descriptive text about the category.

Part

- `partID`: Primary key, unique identifier for each part.
- `name`: Name of the part.
- `description`: Descriptive text about the part.
- `price`: Price of the part.
- `categoryID`: Foreign key, links part to its category.

Sale

- `saleID`: Primary key, unique identifier for each sale.
- `saleDate`: Date when the sale was processed.
- `workerID`: Foreign key, links sale to the worker who processed it.

Worker

- `workerID`: Primary key, unique identifier for each worker.
- `name`: Name of the worker.
- `role`: Role of the worker within the shop.

Monthly Report

- `reportID`: Primary key, unique identifier for each report.
- `month`: Month of the report.
- `totalSales`: Total number of sales in that month.
- `totalRevenue`: Total revenue generated in that month.

Stock Status

- `partID`: Foreign key, links to the part being tracked.
- `stockLevel`: Current stock level of the part.
- `lastRestockDate`: The last date when the stock was replenished.

This detailed documentation will guide the development and implementation of your database system, ensuring all requirements are met and providing a clear understanding of each component.

3. Database Logical Model (SQL Code of Your Project)

The logical design of the database transforms the conceptual design into a structure that can be implemented using SQL. This section details the SQL code for creating tables based on your ERD, including primary keys, foreign keys, and other constraints that ensure data integrity and support the relationships between tables.

SQL Database Logical Design

Here is the SQL Data Definition Language (DDL) code for defining the tables as per the enhanced ERD discussed earlier.

```
-- Creating the Category Table
CREATE TABLE Category (
    categoryID INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    description VARCHAR(255)
);

-- Creating the Part Table
CREATE TABLE Part (
    partID INT PRIMARY KEY,
    serialCode VARCHAR(100) NOT NULL,
    partName VARCHAR(100) NOT NULL,
    price DECIMAL(10,2) NOT NULL,
    description TEXT,
    stockLevel INT NOT NULL,
    availabilityStatus ENUM('Available', 'Not Available') NOT NULL,
    categoryID INT,
    FOREIGN KEY (categoryID) REFERENCES Category(categoryID)
);
```

```
-- Creating the Worker Table
CREATE TABLE Worker (
    workerID INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    shift ENUM('Morning', 'Evening') NOT NULL,
    role VARCHAR(50) NOT NULL
);

-- Creating the Sale Table
CREATE TABLE Sale (
    saleID INT PRIMARY KEY,
    saleDate DATE NOT NULL,
    totalPrice DECIMAL(10,2) NOT NULL,
    workerID INT,
    FOREIGN KEY (workerID) REFERENCES Worker(workerID)
);
```

```
-- Creating the Monthly Report Table
CREATE TABLE MonthlyReport (
    reportID INT PRIMARY KEY,
    month DATE NOT NULL,
    totalSales INT NOT NULL,
    totalRevenue DECIMAL(10,2) NOT NULL
);

-- Creating the Stock Status Table
CREATE TABLE StockStatus (
    statusID INT PRIMARY KEY,
    partID INT,
    currentStock INT NOT NULL,
    lastRestockDate DATE,
    FOREIGN KEY (partID) REFERENCES Part(partID)
);
```

```
-- Creating the Sales-Parts Many-to-Many Relationship Table
CREATE TABLE SaleParts (
    saleID INT,
    partID INT,
    quantity INT NOT NULL,
    FOREIGN KEY (saleID) REFERENCES Sale(saleID),
    FOREIGN KEY (partID) REFERENCES Part(partID),
    PRIMARY KEY (saleID, partID)
);
```

Explanation

1. Category Table:

- Contains information about different categories of parts.

2. Part Table:

- Stores data about each part including its price, stock level, and which category it belongs to.

3. Worker Table:

- Includes details about employees, such as their shifts and roles within the shop.

4. Sale Table:

- Records details about sales transactions including the worker who handled the sale.

5. Monthly Report Table:

- Aggregates sales data into monthly reports showing total sales and revenue.

6. Stock Status Table:

- Tracks the availability and stock levels of each part to manage inventory effectively.

7. SaleParts Table (Many-to-Many Relationship):

- Facilitates the many-to-many relationship between sales and parts, including the quantity of each part sold per sale.

Normalization of the Database

Normalization is the process of organizing data in a database to reduce redundancy and improve data integrity. The aim is to divide large tables into smaller, more compact tables while retaining relationships between them. Here, I'll detail how the tables from your database design can be normalized up to the Third Normal Form (3NF), which is typically sufficient for most applications to ensure data integrity without unnecessary redundancy.

Explanation and Application of Normal Forms:

1. First Normal Form (1NF)

- **Rules:** Ensure all table columns contain atomic (indivisible) values and that each record is unique.
- **Application:**
 - **Category, Part, Worker, Sale, MonthlyReport, StockStatus:** All these tables already adhere to 1NF as they have primary keys (`categoryID`, `partID`, `workerID`, `saleID`, `reportID`, `statusID` respectively), and all attributes store single (atomic) values.

2. Second Normal Form (2NF)

- **Rules:** Meet all the requirements of 1NF, and ensure that all non-key attributes are fully dependent on the primary key.
- **Application:**
 - **SaleParts Table:** This table is in 2NF because it contains composite primary keys (`saleID`, `partID`), and other attributes like `quantity` are dependent on the composite key, not just part of it.
 - All other tables have all non-key attributes dependent on the primary key, confirming adherence to 2NF.

3. Third Normal Form (3NF)

- **Rules:** Meet all the requirements of 2NF, and ensure that there are no transitive dependencies (i.e., non-key attributes should not depend on other non-key attributes).
- **Application:**
 - **Worker Table:** Initially, if `shift` and `role` attributes were determined by `name`, it would violate 3NF. However, since each worker ID is unique and other attributes do not depend on each other but only on `workerID`, this table is in 3NF.
 - **Part Table:** The `categoryID` links to `Category`, but all other attributes such as `description`, `price`, and `stockLevel` depend only on `partID`, ensuring the table is in 3NF.
 - **Monthly Report** might appear to have potential transitive dependencies if total values were calculated from other data within the table. However, it stores totals directly related to the `reportID` and does not derive them from other attributes in the table, ensuring compliance with 3NF.

Relational Database Schemas (after Normalization)

After ensuring the database tables adhere to the third normal form (3NF), here's a detailed description of each table, referred to as the relational database schema. This schema includes all tables defined earlier, structured to prevent redundancy and promote data integrity.

1. Category Table: Stores different categories of car parts. Each category is uniquely identified by `categoryID`.

2-Part Table: Each part has a unique serial code and is linked to one category. Details like price and stock level are directly related to each part.

3-Worker Table: Information about employees including their working shift and role. Each worker has a unique identifier.

4. Sale Table: Records details of sales transactions. Each sale is processed by a worker.

5. Monthly Report Table: Aggregates data from sales over each month, providing totals for sales and revenue.

6. Stock Status Table : Tracks the current stock and restock dates for parts. Linked to the parts to maintain real-time inventory updates.

7. SaleParts Table (Many-to-Many Relationship between Sale and Part): Facilitates the many-to-many relationship between sales and parts, including the quantity of each part sold per sale.

Normalization Confirmation

- **All tables are designed to eliminate redundancy**, ensuring that non-key attributes depend only on the primary key.
- **Foreign key constraints** are used to maintain referential integrity across related tables.
- **Primary keys** ensure uniqueness within each table, facilitating fast and efficient querying.

SQL Statements (DDL & DML):

DDL Statements

DDL statements are used to define the database structure. This includes creating tables, setting up primary and foreign keys, and defining constraints.

```
CREATE TABLE Category ( categoryID INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(100) NOT NULL, description
```

```
VARCHAR(255) );
```

```
CREATE TABLE Part ( partID INT AUTO_INCREMENT PRIMARY KEY, serialCode VARCHAR(100) NOT NULL UNIQUE, partName
```

```
VARCHAR(100) NOT NULL, price DECIMAL(10,2) NOT NULL, description TEXT, stockLevel INT NOT NULL, availabilityStatus
```

```
ENUM('Available', 'Not Available') NOT NULL, categoryID INT, FOREIGN KEY (categoryID) REFERENCES Category(categoryID) ); –
```

```
Create the Worker table CREATE TABLE Worker ( workerID INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(100) NOT
```

```
NULL, shift ENUM('Morning', 'Evening') NOT NULL, role VARCHAR(50) NOT NULL );
```

```
CREATE TABLE Sale ( saleID INT AUTO_INCREMENT PRIMARY KEY, saleDate DATE NOT NULL, totalPrice DECIMAL(10,2) NOT
```

```
NULL, workerID INT, FOREIGN KEY (workerID) REFERENCES Worker(workerID) );
```

```
CREATE TABLE MonthlyReport ( reportID INT AUTO_INCREMENT PRIMARY KEY, month DATE NOT NULL, totalSales INT NOT NULL,
```

```
totalRevenue DECIMAL(10,2) NOT NULL );
```

```
CREATE TABLE StockStatus ( statusID INT AUTO_INCREMENT PRIMARY KEY, partID INT, currentStock INT NOT NULL,
```

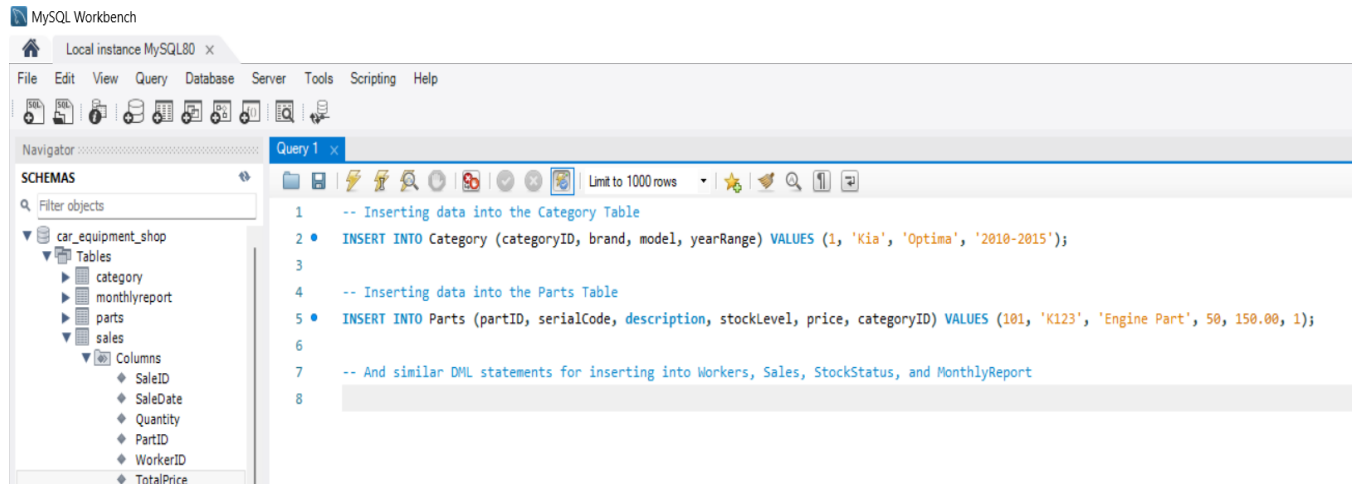
```
lastRestockDate DATE, FOREIGN KEY (partID) REFERENCES Part(partID) );
```

```
– Create the SaleParts table for many-to-many relationship between Sale and Part CREATE TABLE SaleParts ( saleID INT, partID
```

```
INT, quantity INT NOT NULL, PRIMARY KEY (saleID, partID), FOREIGN KEY (saleID) REFERENCES Sale(saleID), FOREIGN KEY
```

DML Statements

DML statements are used to manipulate the data within the tables, such as inserting initial data, updating records, or deleting entries.



Conclusion and Next Steps

With the schemas and SQL scripts, your database is ready for implementation. You can begin creating tables in your database environment, populating them with data, and testing interactions such as transactions and report generation. If any adjustments or additional functionalities are required, you can update the schemas and scripts accordingly.